Assignment 8 Observe Linux Pages and Kernel Allocators

Linux Systems and Performance — Week 8

Build faultwatch and kmemwatch, then extend your work into a BPF-based slab monitor.

1 Assignment Philosophy

Build tools that reveal when virtual memory becomes resident, how page tables consume memory, and how the kernel allocates pages and slab objects. Validate your output against Linux and explain the evidence. A working display alone is not sufficient.

BUILD → OBSERVE → VALIDATE → EXPLAIN

In Week 1, mytop turned kernel process information into periodic CPU measurements. Here, myslabtop turns allocator snapshots and BPF events into periodic memory and activity measurements. Existing tools are validation references; inventing a new tool category is not the goal.

Level	Build	Observe and validate	Main question
1	faultwatch	Your memory experiments; procfs and time/perf	When are pages populated, and what do page tables cost?
2	kmemwatch	Buddy, zone and slab snapshots	Where is kernel memory held?
3	myslabtop	Slab snapshots plus four BPF events	Which caches are large, and which allocation paths are busy?

1.1 Student Implementation and Reuse

No instructor-provided case binaries, workload programs, or assignment starter code are required. Write your own memory experiment program and workload scripts. Reuse that program across experiments and reuse the Level 2 slab parser in Level 3. A single experiment program with several modes is encouraged.

AI coding tools may be used. You must understand, debug, modify, and explain the submitted code. All reported measurements must come from your own runs. Credit depends on implementation, experiments, validation, and explanation together.

1.2 Common Rules

- Use the reference VM profile in Appendix A with root access for the full assignment. An alternative environment is acceptable only if it passes all preflight checks and matches the documented hook prototypes; explain any differences. Do not change tracing or cache settings on a shared or production system.
- Record kernel release, architecture, page size, RAM, THP policy, compiler, and BPF tool versions. Appendix A defines the BPF reference interface.
- Implement user-space tools in a language available in the course environment. Level 3 BPF programs use C with libbpf and BTF/CO-RE; the user-space loader uses libbpf. No kernel modification is required.
- Do not invoke existing monitoring tools to generate your required output. Use them only for validation. Standard Linux APIs and libraries are permitted.
- Use runtime page size; do not assume fixed zones, orders, or cache names. Report bytes or KiB explicitly; procfs fields labelled kB are treated as KiB.
- Keep each workload below 25% of RAM, including all workers. Reduce stated sizes if necessary and document the change. Avoid swap activity and OOM; memory-pressure recovery is Week 9 material.
- Record exact commands and workload sizes. Repeat each required workload experiment three times in fresh runs and report a median or range. Baseline and permission-handling demonstrations need only one run. See Section 7.1 for the Level 3 interval comparison and evidence-reuse rules. Use deltas around checkpoints to separate workload effects from process startup.
- Deadline: before the Week 9 class begins.

2 Level 1 Build faultwatch

2.1 Required Interface

```
./faultwatch [--range START-END] [--pfn] [--interval SEC] PID
./faultwatch [--delay SEC] [--interval SEC] -- COMMAND [ARGS...]
```

PID mode observes an existing process. Command mode starts a child, samples after a default delay of one second if the child is still alive, and reports exit-time minor and major fault totals from wait4(). Delay is an observer delay, not an automatic stop of the child. Experiments requiring checkpoints must explicitly wait for input in the workload. Short-lived children must still produce an exit report. Without --interval, produce one snapshot when possible. --range and --pfn are required only in PID mode; reject them in command mode with a clear message.

Option	Required behavior
--range START-END	Hexadecimal, page-aligned, half-open interval [START, END); classify every base page
--pfn	With --range, show PFNs and the number of distinct frames when access permits
--interval SEC	Positive sampling period; show totals and changes between samples, with elapsed time
--delay SEC	Nonnegative initial observation delay in command mode

2.2 Required Data and Error Handling

- Parse minflt and majflt from /proc/PID/stat, and VmRSS, RssAnon, RssFile, RssShmem, VmPTE from status. The stat command field can contain spaces and parentheses; find its final closing parenthesis before parsing subsequent fields.
- Sum Pss, Private_Dirty, Shared_Clean, and AnonHugePages from smaps for a process-wide report. Keep this scope distinct from a selected address range. Report absent fields and permission failures explicitly.
- For --range, use pread() or equivalent on pagemap: one 64-bit entry at offset (virtual_address / page_size) × 8. Bit 63 is present, 62 swapped, 56 exclusive, and bits 0–54 contain PFN when permitted.
- Use disjoint classes: present, swapped, and neither present nor swapped. Report the exclusive bit count separately for present entries. A zero entry does not distinguish unmapped memory from every kind of untouched mapping.
- PFN disclosure normally requires CAP_SYS_ADMIN in the initial user namespace. Root inside a restricted container may be insufficient. Do not interpret masked zero PFNs as evidence of shared frames; report PFN analysis as unavailable if access is insufficient.
- Handle disappearing processes, short reads, invalid ranges and missing permissions. Label the first interval sample as a baseline. In PID mode, Ctrl-C stops the observer and leaves the target running. In command mode, place the launched workload in a separate process group; on Ctrl-C, terminate that group, use a bounded wait followed by SIGKILL if necessary, and reap the direct child. Record the timeout. End sampling after child exit.

2.3 Output Example

```
PID 1234  elapsed 1.002 s
minflt_total 420  minflt_delta 256  majflt_total 0  majflt_delta 0
VmRSS_KiB 2100  RssAnon_KiB 1200  VmPTE_KiB 48
range [0x...,0x...): present 256  swapped 0  neither 0  exclusive 256
```

Values above are illustrative, not expected results. The first sample has N/A deltas. Also print the other required fields; output layout is flexible if labels and units are explicit.

2.4 Workload Requirements

Print the PID, mapped region addresses, region lengths, and checkpoint names. Wait for a line of input between phases so observations can finish. Disable THP on anonymous test regions with MADV_NOHUGEPAGE, check the return value, and record the setting. Keep inspection code and other activity small and account for their background faults.

For file experiments, use a fully written, non-sparse file rather than an unwritten or sparse file. Keep it on a disk-backed filesystem, record the filesystem, and prepare it before the timed access. A tmpfs file is not an appropriate substitute for the cold disk-backed experiment.

3 Level 1 Experiments and Explanation

3.1 Required Experiments

Experiment	Workload and checkpoints	Explain
Demand paging	Map 64 MiB anonymously; inspect before touch, after one write per page, and after writing every byte	Reservation versus residency; pages versus bytes
Read then write	Fresh 64 MiB region; read one byte per page, then write one byte per page; prevent the compiler from removing reads	Zero-page behavior; present state versus Rss; first-write faults
Fork and COW	Parent writes 64 MiB, forks; inspect both, then child writes 16 MiB and waits	Rss, Pss, exclusive bits and child minor faults
Cold and warm file mapping	Map a prepared 64 MiB file, read every page; unmap and map again for a warm run; repeat the pair with MADV_RANDOM	Major/minor faults, readahead and fault-around
Page-table footprint	In separate fresh runs, write each page of a dense 1 MiB region, or one byte every 2 MiB across 512 MiB	Similar resident data with different page-table cost

For the cold file run on a dedicated VM, prepare the file, run sync, then use root to write 3 to /proc/sys/vm/drop_caches. Do not drop caches for the warm run. Remapping is essential: rereading an already populated mapping is not the same test as faulting a new warm mapping. Describe cold-cache control limits and explain any difference from a one-fault-per-page prediction. Such differences are acceptable when supported by evidence.

The sparse geometry above targets 4 KiB base pages on x86-64. On another architecture/page size, adapt the spacing to the span of one lowest-level page-table page and explain the geometry. Record addresses and alignment; boundary crossings affect the estimate.

3.2 Fault Accounting Table

For every row, measure faults as the increase in the observed process between the two checkpoints bracketing that phase, not the lifetime total. Measure Rss and VmPTE differences over the same boundaries. For COW use the child immediately before and after its writes, while the parent stays alive without writing. For dense/sparse runs compare a pre-mmap baseline with the touched state. For cold/warm file runs compare immediately before and after the page-reading loop on each new mapping. Predict minor faults before running. Use the median of three runs and explain discrepancies; file experiments must include measured major faults as well.

Phase	Pages touched	Predicted minor	Measured minor/major	ΔRss KiB	ΔVmPTE KiB
Demand write					
Write every byte after page touch					
First read					
Write after read					
Child COW writes					
Default cold file					
Default warm remapping					
RANDOM cold file					
RANDOM warm remapping					
Dense mapping					
Sparse mapping					

3.3 Required Explanation

1. Which Week 7 hypotheses about Size versus Rss were confirmed or revised?

2. Can an anonymous read-touched page be present without the Rss increase expected from a private written page? Explain your evidence and any PFN-access limitation.

3. How do fork and later writes change sharing, Pss, and exclusive-page observations? State the limits of treating one pagemap bit as a complete ownership test.

4. Why can a newly mapped warm file produce minor faults, and why can faults be fewer than base pages?

5. Estimate dense and sparse page-table costs from the actual geometry. Which page-table level explains the main difference?

4 Level 2 Build kmemwatch

4.1 Required Interface

```
./kmemwatch [--buddy] [--watermarks] [--migrate]
            [--slab CACHE] [--top N] [--diff SEC]
```

Option	Required output
--buddy	Every node/zone and order: free block count, KiB, cumulative KiB in this order or larger, largest available order
--watermarks	Per-zone free pages and min/low/high thresholds; comparisons in consistent units
--migrate	Per-node/zone/migratetype blocks per order from pagetypeinfo, when readable
--slab CACHE	Cache geometry and current object/slab counts
--top N	Largest N caches by estimated cache memory
--diff SEC	Two snapshots separated by a positive interval; changed entries sorted by absolute memory change within buddy and slab sections

With no display option, show --buddy --watermarks --top 10. --diff alone uses that same selection. When display options are present, show only those sections plus meminfo totals. --slab and --top may be combined; avoid duplicate cache rows. In a diff, compare the union of entries from both snapshots, treating an absent entry as zero for the change and marking it added/removed. For --top with --diff, rank changed caches across the full cache set before selecting N. Watermark and migratetype changes use their native counters and are shown in separate labelled sections. Reject nonpositive intervals or N.

Read buddyinfo, zoneinfo, pagetypeinfo, slabinfo and meminfo directly. Order n contains blocks of 2^n base pages. A count is a number of blocks, not pages. Reuse the slab parser and derived fields in Level 3.

4.2 Slab and System Fields

For each cache report name, active/total objects, object size, objects per slab, pages per slab, active/total slabs, and estimated memory = total_slabs × pages_per_slab × page_size. Report object utilization as active_objects / total_objects; show N/A for a zero denominator.

Also report Slab, SReclaimable, SUnreclaim, PageTables and KernelStack from meminfo. Label estimated cache footprint separately from object payload and system Slab. Nominal slab geometry is an estimate; allocator configuration and order fallbacks can prevent exact equality.

Use slabinfo for the full required run. In a separate unprivileged run, try readable sysfs slab attributes and report unavailable counts as N/A, not zero. The unprivileged run may have fewer fields than the root run; N/A with a clear reason is acceptable. If enumerating sysfs aliases, resolve symbolic links and avoid counting one underlying cache twice. Missing migratetype data must not stop the other sections.

4.3 Required Experiments

Experiment	Workload	Evidence
Baseline	No additional workload	Zones, orders, thresholds and top caches
Allocate and free	Touch 128 MiB; wait; unmap	Before/during/after buddy snapshots
Partial release	Touch 128 MiB with THP disabled; MADV_DONTNEED every other base page, then the remaining pages	Rss and buddy changes; optional permitted PFN evidence
Process activity	Create 100 sleeping processes; wait; terminate and reap them	Slab, PageTables, KernelStack and cache changes

Experiment	Workload	Evidence
File metadata	Create and stat 10,000 empty files in a fresh directory; delete them afterward	Cache growth and post-delete state
Permission handling	Run kmemwatch without root	Available data, explicit N/A and continued operation

Reduce worker/file counts if system limits require it; record actual successful counts. Release workloads before repeats. Do not infer physical alternating buddies merely from alternating virtual addresses. System background work and physical placement can change high-order behavior. No particular high-order change is required; explain the observed pattern and the limits of your evidence.

5 Connect Process Memory and Allocator State

5.1 Connection Experiment

Use the Level 1 memory program with a 128 MiB anonymous region and THP disabled. At each checkpoint, collect faultwatch and kmemwatch snapshots close together and record timestamps. Do not launch a new workload for every observation.

```
./faultwatch --interval 1 PID
./kmemwatch --buddy --watermarks --top 10
```

Checkpoint	minflt count	Rss KiB	Buddy free KiB	Order ≥9 free KiB
A After mmap, no touch				
B After one write per page				
C After releasing alternate pages				
D After munmap				

Order ≥9 is an order threshold, not an architecture-independent 2 MiB threshold. Convert it using your page size and explain it. A fault count is a count, not KiB.

5.2 Slab Geometry Table

For task_struct, dentry, anon_vma, kmalloc-64 and kmalloc-1k, record the fields below. If a cache is absent or merged, show the actual cache/alias or explain why no entry is available. Do not hard-code this list into the monitor.

Cache	Object B	Objects/slab	Pages/slab	Total slabs	Estimated KiB
task_struct					
dentry					
anon_vma					
kmalloc-64					
kmalloc-1k					

5.3 Required Explanation

1. Convert an order-3 block to bytes on your machine. What physical contiguity does the order encode?
2. Why can mmap reserve a large region without a matching buddy decrease? Why might touch-time buddy change differ from ΔRss?
3. What conditions let two free buddies merge? What can and cannot be concluded from the partial-release experiment?
4. Where are zone free pages relative to min/low/high? Predict what falling below low would imply; do not deliberately create pressure.
5. Compare Slab, SReclaimable and SUnreclaim. Why does freeing an object not guarantee immediate release of a slab page?
6. Show two categories of kernel memory excluded from a process's Rss and identify the counters used to observe them.

5.4 Reuse in Level 3

One checkpoint workload may satisfy both the Level 2 allocation/partial-release experiments and Section 5.1. Keep the process and file workload scripts for Level 3. Add phase markers and timestamps if needed. No new instructor workload is supplied. In Level 3 the same workloads connect a change in cache state to the allocation and free events observed during that interval.

6 Level 3 Build myslabtop

6.1 Goal and Required Interface

Implement core slabtop-style cache information and add BPF activity from four specified kmem events. Reuse Level 2 parsing; the additional work is BPF collection, interval aggregation, periodic display and validation. A plain terminal table is sufficient; ncurses is optional.

```
sudo ./myslabtop --interval 2 --top 10 --sort size
sudo ./myslabtop --interval 2 --top 10 --sort alloc
sudo ./myslabtop --interval 2 --view kmalloc
```

Support --interval (default 2 seconds), --top (default 10), --sort (default size in cache view) and --view (cache by default, or kmalloc). In kmalloc view sort by allocation rate; --sort alloc is accepted and other explicit sort values must be rejected. Numeric sorts are descending, name sort is ascending, and ties use ascending cache name or requested size. Invalid values must produce an actionable error. Ctrl-C must detach programs and close resources. Do not persistently pin programs/maps by default.

6.2 Cache View

Show summary totals for active/total objects and slabs, object utilization, and the slab ratio reported by the source. In the reference Linux 6.8 SLUB implementation, active_slabs equals total_slabs, so this ratio is normally 100% and does not measure object occupancy. Preserve that distinction in labels and interpretation; report N/A for a zero denominator. Show meminfo Slab, SReclaimable and SUnreclaim as system physical-memory counters. Do not call object payload bytes physical slab memory.

Per-cache field	Source or definition
NAME, OBJS, ACTIVE, USE%	slabinfo name, total objects, active objects, active/total ×100
OBJ SIZE, SLABS, OBJ/SLAB, PAGES/SLAB	slabinfo geometry and total slabs
CACHE KiB	Level 2 nominal footprint estimate
ALLOC/s, FREE/s	Observed successful kmem_cache_alloc and valid kmem_cache_free events / measured elapsed seconds

Support cache sorting by size, objects, use, name, alloc and free. Use NAME as a stable tie-breaker. Size/object/use sorts include snapshot caches even when they have no events. Include event-only cache rows with snapshot fields N/A if the sources cannot be matched.

Label activity columns CACHE_ALLOC_EVT/s and CACHE_FREE_EVT/s (ALLOC/s and FREE/s are permitted if a visible legend defines their scope). They count the two named-cache tracepoints, not all operations affecting that cache. For example, an object allocated through kmem_cache_alloc can be freed through kfree, which does not generate a kmem_cache_free event in the reference kernel. Do not diagnose a leak from unmatched column rates.

Read cache names from the BPF cache pointer, not by guessing from object size. Match canonical names where possible; explain aliases and unmatched names. Only print zero activity when the relevant hooks are attached and collection succeeded. Otherwise print N/A/error. Mark initial rate samples as warming up until a full interval is observed.

6.3 Display Example

```
NAME     OBJS ACTIVE USE% OBJ_B SLABS OBJ/SLAB PAGES/SLAB CACHE_KiB ALLOC/s FREE/s
dentry    ...  ...    ...  ...   ...    ...       ...        ...       ...     ...
Legend: ALLOC/s and FREE/s count only kmem_cache_alloc/free events.
```

Also show the required summary counters. Values and rankings vary by environment. A cache with no observed activity can still occupy memory.

6.4 kmalloc View

Use exact requested byte size as the grouping key. Show successful allocations/s, requested KiB/s, actual allocated KiB/s, and their difference. Sort by allocation rate and show the top N sizes. Display valid kfree events/s separately as a global count: the free event does not carry allocation size or cache identity.

The allocated-minus-requested rate is observed allocation rounding/overhead at these events. It is not total slab fragmentation or retained memory. Large kmalloc requests may involve page allocations; do not assume every event belongs to a small-object slab cache.

Appendix A defines the four hooks and field access. Pointer matching, live-byte reconstruction, PID ownership accounting and leak detection are optional, not required.

7 Level 3 Experiments and Interpretation

7.1 Required Experiments

Use Level 2 workload scripts. At the default 2-second interval, each trace must include at least 3 full intervals before the workload, 4 during workload activity, and 3 after cleanup. Record monotonic timestamps and phase markers. Pace or repeat create/stat/delete and process start/reap operations so activity spans the required intervals; record batch sizes and the actual total operations, which may exceed the nominal 10,000 files or 100 processes. Never leave more than the chosen process limit running at once. Keep separate markers for creation, lookup, deletion, start and reap where applicable. Mark workload phases and collect both views in each experiment; simultaneous reporting or repeated equivalent runs are acceptable if documented.

Experiment	Required observations
Idle baseline	Large caches versus active allocation paths; background variation
Files	Create, stat and delete 10,000 files; follow usage and event rates through all phases
Processes	Create and reap 100 sleeping processes; compare cache activity, footprint and system counters

Repeat each default-interval workload three times, collecting both views either together or in matched runs. One combined collection can serve both Level 2 and Level 3 if all required fields are retained; the two views do not require extra repetitions when collected together. For one selected workload, add one 0.1-second run using the same workload parameters and observation duration as a selected 2-second run; this interval comparison need not be repeated three times. Record how you reset or account for residual caches between runs.

For that selected workload, compare 0.1-second and 2-second intervals. Explain rate variability, responsiveness, and monitoring overhead. The highest-ranked cache can differ across kernels and filesystems; explain your own ranking rather than reproducing a prescribed one.

7.2 Validation

Compare cache snapshots with **slabtop -o** taken close in time. Select at least three caches and compare counts, utilization, geometry and the stated footprint formula. Exact physical-memory equality is not expected from nominal geometry.

Use an existing BPF tool such as a compatible slabratetop, or an independent tracefs/perf event count, to cross-check allocation activity during a short run. State whether it counts attempted function calls or successful events, whether it covers the same allocation paths, and whether it uses the same time window. A kprobe counter on one function is not automatically an exact reference for all tracepoint events.

For every hook, demonstrate successful attachment and an increasing event counter during a workload, or document a zero count with an independent reference that also observed zero. Compare all four raw event counts over equivalent windows; separately compare the success/sentinel-filtered counts used in the display. Check kmalloc requested/allocated byte sums against a short independently captured event sample. A comparison covering only kmem_cache_alloc is insufficient.

Attach one comparison table with one row per event plus requested/allocated byte totals: your value, reference value, interval/scope, filtering rules, and explanation of differences. Small timing-window differences are acceptable when quantified; do not use exact equality across separate runs as the criterion. Existing monitors may validate output but may not supply the program's required values. Consulted or adapted code must be attributed and explained.

7.3 Required Explanation

1. Is the largest cache also the cache with the highest allocation rate? Support the answer with your measurements.
2. Why can high allocation and free rates coexist with nearly constant cache size?
3. Why can cache footprint stay elevated after objects are freed or files are removed?
4. Why is allocations minus frees since attachment not the total number of live objects? Can it be negative?
5. What do requested bytes and allocated bytes mean? Why is their difference not a measurement of all wasted slab memory?
6. Which paths are outside the four-hook scope? Why are mm_page_alloc or vmalloc observations not interchangeable with slab-object observations?
7. How do interval length, concurrent map updates and collector overhead affect results? What collection errors did you detect?
8. Which conclusion would have been unsupported with only snapshots, or only event rates?

7.4 Implementation Quality

Keep cumulative raw counts for each of the four events as validation diagnostics, as well as the filtered counts and kmalloc byte sums used in the display. Export these diagnostics at start/end or on exit with timestamps. Aggregate in BPF maps instead of printing every event. Make concurrent counter updates correct (for example, per-CPU counters reduced in user space). Prefer cumulative counters and user-space deltas over reading and then clearing a live map without synchronization. Document timing skew: a map scan is not a single globally atomic snapshot.

Bound map capacity and cache-name length, and report failed insertions/reads or truncation rather than silently claiming complete data. Report hook attachment failures explicitly; a partial collector must not present a complete four-hook result. Keep resource usage and experiment duration bounded.

8 Submission and Evaluation

8.1 Submission Package

```
hw8_NAME/
  level1/       faultwatch source and memory experiment source
  level2/       kmemwatch source
  level3/       myslabtop user-space and BPF source
  common/       shared code, if used
  workloads/    student-written workload scripts
  logs/         selected raw data and phase timestamps
  Makefile      or documented build scripts
  README.md     environment, build, run and reproduction steps
  report.pdf    or report.md
```

Submit source for every student-created experiment and script. Build dependencies such as vmlinux.h or generated libbpf skeleton headers may be generated locally; document their exact generation commands. Using bpftool-generated .skel.h is permitted and is distinct from instructor-provided assignment starter code.

The report should be concise (recommended 8–10 pages, with selected raw evidence in appendices). Organize it by level. Include the fault accounting, slab geometry and connection tables; Level 3 time series; reference comparisons; and answers to the numbered explanation questions. Record missing permissions and unavailable fields honestly.

Include an AI-use statement if applicable: tools used, their purpose, and at least one suggestion or code issue that you verified, changed or rejected.

8.2 Evaluation

Category	Weight	Evidence
Implementation	35%	Correct collection and calculations across all three levels; BPF map safety; error handling
Experiments and validation	30%	Reproducible commands, predictions, repeats, phase measurements, independent comparisons
Technical explanation	25%	Paging, page tables, buddy/slab behavior, snapshot versus event reasoning
Code and report	10%	Reuse, clarity, reproducibility, attribution and AI-use statement

Absolute counts, memory sizes and cache rankings are not graded across machines. Working code without understanding or validation cannot receive full credit. Optional features do not substitute for required correctness and analysis; this specification assigns no separate bonus points.

8.3 Final Check-Off

- Predict pagemap state, Rss and faults before reading or writing a fresh anonymous region.
- Estimate sparse page-table cost and explain a discrepancy.
- Convert one buddy order and explain its merge condition.
- Show kernel memory that does not belong to process Rss.
- Explain one BPF hook argument and trace it to an output column.
- Change the display interval or workload count and predict the effect before running.
- Explain why one cache is large but quiet, or small but active.
- Demonstrate termination and explain what happens when a hook or map operation fails.

Suggested sequence: Days 1–2 Level 1; Days 3–4 Level 2 and the connection experiment; Days 5–6 Level 3; Day 7 validation and report. Reuse code and workloads throughout.

Appendix A BPF Collection Contract

A.1 Reference Target and Attachment

Reference profile: Ubuntu 24.04 LTS x86-64 VM, 4 KiB base pages, Linux 6.8 with SLUB and kernel BTF, at least 2 vCPUs and 4 GiB RAM. WSL 2 is also allowed, provided its actual kernel supports the required features, all four raw tracepoint prototypes match this contract, and all preflight and attachment checks pass. Record both the WSL version and uname -r. WSL 1 is not an accepted environment for this assignment. For file experiments under WSL 2, use the Linux filesystem inside the distribution rather than a Windows-mounted path such as /mnt/c, and record the filesystem. If the required tracing features are unavailable, use a compatible Linux VM. The hook contract below is defined against upstream Linux v6.8; a distribution kernel must pass the prototype and attachment checks rather than being assumed identical. Use this profile for the full assignment. A different environment must pass the same checks; contact the instructor before proceeding if it does not. Record its exact uname -r; do not infer compatibility from the distribution name alone. Porting across arbitrary kernels is not required.

Use libbpf raw tracepoint attachment with SEC("raw_tp/EVENT") for all four hooks. Raw tracepoints expose the original TP_PROTO arguments as ctx->args[], not the formatted trace record fields. Do not replace this with an ordinary tracepoint context layout. Use BTF/CO-RE and safe kernel-read helpers for kernel structure fields; do not hard-code structure offsets.

Event label	BPF section	Required raw arguments, zero-based
kmem:kmem_cache_alloc	raw_tp/kmem_cache_alloc	args[1]: returned object pointer; args[2]: struct kmem_cache *
kmem:kmem_cache_free	raw_tp/kmem_cache_free	args[1]: object pointer; args[2]: const struct kmem_cache *
kmem:kmalloc	raw_tp/kmalloc	args[1]: returned pointer; args[2]: size_t bytes_req; args[3]: size_t bytes_alloc
kmem:kfree	raw_tp/kfree	args[1]: object pointer

In these prototypes args[0] is unsigned long call_site. The allocation prototypes also contain flags/node arguments not needed for required aggregation. On this 64-bit target, pointers and size_t values are carried in 64-bit slots. Convert to the proper type before use.

For the two cache hooks, let s be args[2]. Read s->name as a kernel string through its pointer using a bounded helper. In the allocation hook, s->object_size is nominal payload size and s->size is allocator object stride including applicable padding/metadata. Use the cache name for required grouping. A size alone does not identify a cache.

Count successful allocations only; reject NULL and zero-size sentinel pointers (the reference kernel's ZERO_OR_NULL_PTR convention rejects pointer values ≤16). For required free counts, exclude only NULL and the documented zero-size sentinels. Full pointer-validity or original-allocation verification is not required. Call these non-sentinel observed free events, not proof that every object lifetime was tracked. Count failed allocations separately if desired. Emit errors for failed field/name reads; do not silently assign them to a real cache.

A.2 Environment and Build Responsibilities

The full run needs root or equivalent BPF/tracing capabilities, a kernel with BPF tracing and BTF enabled, a clang build with the BPF target, libbpf development files, libelf/zlib build dependencies, and bpftool. The assignment assumes prior instruction on the BPF lifecycle and maps.

Generate type declarations from /sys/kernel/btf/vmlinux using bpftool btf dump file /sys/kernel/btf/vmlinux format c. Compile the student BPF source with clang -target bpf -O2 -g and appropriate include paths. Load and attach it using libbpf. Exact file names and Makefile structure are student choices. If using generated skeleton headers, generate them from your compiled BPF object with bpftool gen skeleton.

The reference package setup uses the commands below. Record the installed package versions; the distribution kernel and package revisions are not pinned by this document. Use the kernel sources matching uname -r to verify the raw argument contract if distribution patches differ.

```
sudo apt update
sudo apt install build-essential clang llvm libbpf-dev libelf-dev zlib1g-dev pkg-config linux-tools-common linux-tools-$(uname -r)
uname -r
uname -m
getconf PAGESIZE
clang --version
pkg-config --modversion libbpf
bpftool version
sudo bpftool feature probe kernel
test -r /sys/kernel/btf/vmlinux
sudo mount -t tracefs tracefs /sys/kernel/tracing
sudo ls /sys/kernel/tracing/events/kmem/{kmem_cache_alloc,kmem_cache_free,kmalloc,kfree}/format
bpftool btf dump file /sys/kernel/btf/vmlinux format c > vmlinux.h
```

Run the commands in bash. Skip the mount command if tracefs is already mounted. Expected preflight results: x86_64, page size 4096, readable BTF, all four format files present, clang with BPF support, and successful attachment of your four programs. The format files confirm event availability; they do not define raw argument order. If a running-kernel tools package or bpftool is unavailable, resolve that environment issue before starting Level 3.

Before experiments, verify all four events exist in tracefs, BTF is readable, the BPF backend compiles, and all four programs attach. Record tool versions and the command sequence. If any requirement fails, report the failed check; do not substitute zeros or silently choose a different hook.

Appendix B Scope and Reference Material

B.1 What the Four Hooks Mean

kmem_cache_alloc/free describe observed object activity at those tracepoints. They do not describe every internal freelist operation or slab page transition. Coverage of bulk and alternate allocation/free paths depends on where the target kernel emits these events; do not claim universal API coverage without verifying it.

kmalloc reports requested and allocated byte counts at its events. It does not directly provide a cache name. Grouping it by request size is a separate view from grouping named-cache events by s->name. Do not combine the two views into an assumed exhaustive total without checking for gaps and overlap.

kfree carries no size or original allocating task. The required implementation counts valid free events globally. Recovering freed bytes requires matching to earlier observed allocations, with explicit handling of pre-existing objects, missing events, pointer reuse and map capacity. That extension is optional.

vmalloc activity, direct page allocation and memory-pressure recovery are outside the required BPF scope. These statistics do not establish total kernel memory use, total live slab bytes, or a memory leak. Nor does the current PID at an event prove ownership of that memory: asynchronous work and frees can occur in a different context.

The snapshot and event sources are collected at nearby times, not atomically. A cache may be absent from one source or disappear between reads. Explain these limits in the report rather than forcing all numbers to agree.

B.2 Independent Event Validation

The default independent reference is perf recording of the same four tracepoints. In a dedicated VM, keep both collectors running while the marked workload executes inside the observation window.

```
sudo perf record -a -e kmem:kmem_cache_alloc -e kmem:kmem_cache_free -e kmem:kmalloc -e kmem:kfree -o kmem.data --
sleep 30
sudo perf script -i kmem.data > kmem.events.txt
sudo perf script -i kmem.data -F trace > kmem.fields.txt
```

Keep the event-labelled output for per-event counts and the field output for kmalloc pointer/bytes_req/bytes_alloc checks. Event field rendering varies by perf version; verify one line for each event before relying on a parser. Explain pointer masking if it prevents matching the BPF success filter. Check for lost records and avoid comparing an incomplete reference trace as if it were exact. Report raw counts separately from pointer-filtered display counts. Use these reference outputs only in validation, never to generate myslabtop output. A compatible alternative is acceptable if it covers all four required checks and explains its semantics.

B.3 Official Sources

Use the target kernel source as the authority for argument order and structure fields. Some narrative tracing documentation describes older prototypes; inspect the pinned source before copying example hooks.

- Linux v6.8 kmem tracepoint definitions: https://github.com/torvalds/linux/blob/v6.8/include/trace/events/kmem.h
- Linux v6.8 slab implementation and structures: https://github.com/torvalds/linux/blob/v6.8/mm/slub.c and https://github.com/torvalds/linux/blob/v6.8/mm/slab.h
- Kernel pagemap documentation: https://docs.kernel.org/admin-guide/mm/pagemap.html
- Kernel kmem event overview: https://docs.kernel.org/trace/events-kmem.html
- libbpf lifecycle, maps and generated skeletons: https://docs.kernel.org/bpf/libbpf/libbpf_overview.html
- slabtop reference: https://github.com/procps-ng/procps/blob/master/man/slabtop.1
- BPF activity reference: https://github.com/iovisor/bcc/blob/master/libbpf-tools/slabratetop.bpf.c

Existing reference implementations may observe a different subset of paths, count attempts instead of successful allocations, or use a different backend. Document these differences when validating; identical tool names or similar columns do not guarantee identical semantics.

Build the collector. Observe the workload. Validate the numbers. Explain the limits.